GUIDA PRATICA / PRACTICAL GUIDE

Gazebo su Parallels

Guida passo passo della configurazione globale applicata sulla VM

Ubuntu 24.04 ARM64 | ROS 2 Jazzy | Gazebo Harmonic

Revisione 4 / Revision 4 - 29 October 2026

Procedura principale	Scopo
Mesa privata + OGRE modificato	Rendering e sensori del caso collaudato
Deleghe gz / gzserver + defaults	Attivazione dai comandi normali
Recupero scena GUI	Visuale LiDAR sul sensore
Backup e rollback	Ripristino degli ingressi originali

Il profilo globale e la PR sono due risultati distinti. Questa guida conserva la procedura del profilo e aggiunge le evidenze successive.

Circa 60 aggiornamenti camera GUI/s osservati. La nuova VM pulita resta da verificare.

Guida passo passo da Ubuntu 24.04 ARM64 con ROS 2 Jazzy a Gazebo Harmonic con camera e LiDAR funzionanti. Revisione 4 del 29 ottobre 2026: configurazione globale della VM.

Questa procedura ricostruisce il workaround verificato su Mac M5 Max, Parallels Desktop/Tools 27.0.3, Ubuntu 24.04 ARM64, Gazebo Sim 8.15.0 e gz-rendering 8.2.3. Non è una patch ufficiale Gazebo o Mesa e non dimostra che tutte le VM Parallels abbiano lo stesso difetto.

Risultato verificato	Misura
GUI 3D accelerata	Circa 60 aggiornamenti camera/s
LiDAR TurtleBot3	360 raggi, circa 10 Hz, distanze corrette
Camera RGB Full HD	Immagini corrette, circa 8-11 immagini/s
Precisione nel test 2D	Errore massimo circa 1,5 mm su 131 raggi

Il kit allegato contiene i sorgenti locali e gli script della configurazione globale usata sulla VM. I comandi quotidiani restano gz sim e ros2 launch: le deleghe installate attivano automaticamente il profilo privato. Non occorre digitare un launcher personalizzato. La copertura riguarda gli ingressi Jazzy indicati al passo 8, non qualsiasi eseguibile o libreria Gazebo incorporata in altri programmi.

Importante per la VM già corretta durante la diagnosi: non ripetere l’installazione. La guida descrive come ricostruire la soluzione partendo da una installazione standard compatibile; la prova completa su una seconda VM pulita resta da eseguire. Sulla VM esistente puoi usare le sezioni di verifica.

- Tutti i comandi di installazione si eseguono nella VM Ubuntu, salvo le operazioni iniziali nell’interfaccia Parallels sul Mac.

- Esegui un blocco alla volta. Controlla il risultato indicato prima di passare al successivo.

- I file di TurtleBot, i mondi e i modelli SDF/URDF non vengono modificati.

COME USARE QUESTA REVISIONE

Questa è la guida della configurazione GLOBALE effettivamente adottata sulla VM, distinta dalla PR #1400. I passi 1-15 restano la procedura principale: Mesa privata, componente OGRE modificato, installazione delle deleghe, libreria dei default e recupero della scena GUI. Le sezioni 16-19 aggiungono motivazioni, nuove evidenze e alternative senza sostituire questo percorso con l'overlay.

Nella cartella trovi gazebo_jazzy_parallels_kit.zip e la sua copia estratta kit/. Trasferisci lo ZIP nella VM per seguire i comandi del passo 4. Il kit scarica Mesa e dipendenze via apt; non è una distribuzione offline di Ubuntu. I sorgenti originali gz-rendering 8.2.3 sono anche conservati in alternativa_overlay_8_2_3/overlay/downloads/gz-rendering-8.2.3.tar.gz: puoi copiarli in kit/build/downloads/ prima della fase source, lasciando invariato il nome, per evitare un nuovo download.

Il risultato di circa 60 riguarda gli aggiornamenti della camera GUI misurati sul sistema descritto. Non garantisce 60 frame realmente presentati al secondo in ogni mondo, né 60 Hz dei sensori. Valuta la correttezza di immagini e scansioni prima della fluidità.

1. CONTROLLARE LA VM E IL PUNTO DI PARTENZA

Sul Mac: usa una VM Ubuntu 24.04 ARM64 su Apple silicon. Prima dell'intervento crea uno snapshot da Parallels, se disponibile nella tua edizione, oppure un backup della VM spenta. Poi avvia Ubuntu e accedi al desktop.

La configurazione collaudata disponeva di 8 vCPU e 16 GB di RAM. Sono le risorse del test, non un requisito minimo dimostrato. Mantieni spazio libero per sorgenti, compilazione e runtime: prevedi almeno alcuni GB.

Parallels usa VirGL/Virtio GPU per l'accelerazione Linux ARM; la disponibilità effettiva va controllata dentro Ubuntu. Usa Parallels Tools coerenti con la versione di Parallels. La combinazione verificata era 27.0.3 build 58680 [2].

Apri un terminale del desktop Ubuntu e controlla sistema e architettura:

```
cat /etc/os-release
uname -m
dpkg --print-architecture
```

Atteso: Ubuntu 24.04, aarch64 e arm64. Se hai amd64, un'altra distribuzione o un'altra versione Ubuntu, non usare questo kit senza adattarlo e collaudarlo.

```
source /opt/ros/jazzy/setup.bash
echo "$ROS_DISTRO"
```

Atteso: jazzy. Se /opt/ros/jazzy/setup.bash non esiste, completa prima l'installazione ufficiale Jazzy per Ubuntu 24.04 [1]. La procedura seguente parte da Jazzy già installato.

2. INSTALLARE GAZEBO E GLI STRUMENTI NECESSARI

Sempre nel terminale Ubuntu. La coppia raccomandata per Jazzy è Gazebo Harmonic, disponibile attraverso i vendor packages del repository ROS. Non è necessario aggiungere un secondo repository Gazebo per questa procedura [3].

```
sudo apt update
sudo apt install ros-jazzy-ros-gz
sudo apt install build-essential cmake pkg-config curl ca-certificates \
  patch unzip python3 python3-pil mesa-utils libogre-1.9-dev \
  libgl-dev qtbase5-dev libprotobuf-dev protobuf-compiler
```

Questi comandi installano Gazebo e dipendenze di compilazione. Il workaround non richiede di retrocedere Mesa di sistema. Le versioni Mesa storiche verranno soltanto scaricate ed estratte più avanti.

```
source /opt/ros/jazzy/setup.bash
command -v gz
gz sim --version
ros2 pkg prefix ros_gz_sim
```

Nell'installazione collaudata: Gazebo Sim 8.15.0 e ros_gz_sim 1.0.24. Il comando gz proviene dal vendor gz_tools di Jazzy.

```
grep "^Version:" \
  /opt/ros/jazzy/opt/gz_rendering_vendor/lib/pkgconfig/gz-rendering8.pc
pkg-config --modversion OGRE
```

Atteso: gz-rendering 8.2.3 e OGRE 1.9.0. Il controllo di gz-rendering legge il file .pc direttamente: il semplice source di ROS può non inserire i pkg-config dei vendor in PKG_CONFIG_PATH.

Se le versioni differiscono, fermati. Non sostituire alla cieca numeri, librerie o controlli hash: la compilazione e l'installazione del kit sono limitate alla combinazione collaudata.

3. VERIFICARE CHE VIRGL SIA REALMENTE ATTIVO

Esegui il controllo dal terminale del desktop Ubuntu, non dal terminale macOS:

```
glxinfo -B
```

Cerca direct rendering: Yes, un renderer che contenga virgl e, quando presente, Accelerated: yes. Sul sistema verificato compariva virgl (Apple M5 Max (Compat)). Il nome del processore grafico cambia sugli altri Mac.

Se compare llvmpipe o softpipe, il rendering sta avvenendo sulla CPU. In quel caso non procedere assumendo che il kit possa attivare una GPU virtuale che la VM non espone.

```
printenv | grep -E \
  "^(LIBGL_|GALLIUM_|MESA_|LD_PRELOAD=|QT_QPA_PLATFORM=)"
```

Su una nuova installazione non dovrebbero esserci override grafici impostati da tentativi precedenti. Se avevi configurazioni personalizzate, controllale prima. Il profilo Gazebo elimina per il proprio processo i comuni override che forzano software rendering o versioni GL; non riscrive la shell globale.

```
sudo dmesg | grep -i virgl
```

È un controllo aggiuntivo suggerito da Parallels [2]. Il test determinante per l’applicazione resta il renderer effettivo mostrato da glxinfo.

Un numero di versione OpenGL alto non basta a garantire che tutti gli shader Gazebo funzionino. Sul caso verificato il driver dichiarava OpenGL 4.0, ma alcuni percorsi OGRE 2 e i passaggi dei sensori restavano problematici.

Se DISPLAY non è disponibile perché stai lavorando via SSH, esegui questo primo controllo nel terminale grafico della VM. Il kit non abilita inoltro X11 e non richiede di renderizzare sul Mac via SSH.

4. COPIARE E APRIRE IL KIT NELLA VM

Scarica gazebo_jazzy_parallels_kit.zip e copialo nella cartella Downloads della VM. Puoi usare una cartella condivisa Parallels o il trasferimento file che preferisci. Il file deve trovarsi nel filesystem Ubuntu prima di eseguire questi comandi.

```
mkdir -p "$HOME/gazebo-parallels"
unzip "$HOME/Downloads/gazebo_jazzy_parallels_kit.zip" \
  -d "$HOME/gazebo-parallels"
cd "$HOME/gazebo-parallels/kit"
sha256sum -c SHA256SUMS
```

Atteso: tutti i file risultano OK. Le checksum controllano l’integrità del contenuto del kit; non costituiscono una firma digitale del produttore.

File o cartella	Scopo
prepare.sh	Preparazione in quattro fasi
sources/	Patch, profilo Ruby, ingressi e libreria dei default
CMakeLists.txt	Compilazione riproducibile del componente
install.py	Verifica e installazione con backup/diversioni
rollback.py	Ripristino degli ingressi originali
tools/ e tests/	Controlli GPU, sensori, distanze e GUI

Torna in questa cartella all’inizio di ogni nuova sessione di lavoro sul kit. I comandi dei passi 5-9 presuppongono questo percorso.

```
source /opt/ros/jazzy/setup.bash
```

Non eseguire prepare.sh con sudo: deve preparare i file nel tuo spazio utente. L’autorizzazione root serve solo al passo esplicito di installazione.

5. PREPARARE MESA PRIVATA, SENZA DOWNGRADE DEL SISTEMA

Esegui soltanto la fase Mesa:

```
bash prepare.sh mesa
```

Lo script controlla Ubuntu 24.04 arm64, gz-rendering 8.2.3 e OGRE 1.9.0. Poi usa apt-get download e dpkg-deb -x. Scarica pacchetti, ma non li registra come installati.

Pacchetto privato	Versione fissata
libegl-mesa0, libgbm1	24.0.5-1ubuntu1
libgl1-mesa-dri, libglapi-mesa	24.0.5-1ubuntu1
libglx-mesa0	24.0.5-1ubuntu1
libllvm17t64	1:17.0.6-9ubuntu1
libxcb-dri2-0	1.15-1ubuntu2

I file .deb rimangono in build/downloads; il contenuto estratto è in build/mesa-root. Le dipendenze LLVM e XCB fanno parte della combinazione verificata: non basta copiare soltanto il driver virtio_gpu_dri.so.

```
dpkg-query -W -f='${Version}\n' libgl1-mesa-dri
```

Annota la versione di sistema. Nella VM verificata è rimasta 25.2.8-0ubuntu0.24.04.4 anche dopo l’attivazione del workaround. Su un’altra installazione può essere differente.

Se apt non trova una versione fissata: controlla prima che le sorgenti ufficiali Ubuntu Noble siano presenti e che apt update sia riuscito. Non sostituire il comando con sudo apt install di una vecchia Mesa e non scegliere versioni casuali. Il kit si deve fermare se i pacchetti necessari non sono disponibili.

6. SCARICARE I SORGENTI E APPLICARE LA PATCH

```
bash prepare.sh source
```

Questa fase scarica il tag ufficiale gz-rendering8_8.2.3, ne verifica lo SHA256 fissato nel kit, estrae i sorgenti in build/src e applica sources/gazebo-virgl.patch.

La patch contiene tre interventi funzionali:

- Forza la disattivazione delle ombre delle luci in OGRE 1. Il workaround combinato ha reso utilizzabile la GUI osservata, ma manca una registrazione A/B controllata che attribuisca la scomparsa del flicker alla sola modifica delle ombre.

- Per il LiDAR a una sola riga conserva il workaround originale a tre righe interne, campionamento centrale e aspect ratio coerente. Le API e i messaggi mantengono le dimensioni richieste. I test successivi mostrano che il padding non è necessario nei casi provati quando le coordinate UV e il campionamento sono corretti; qui è mantenuto perché questa guida ricostruisce il profilo globale adottato.

- Include il ripristino dell'orientamento delle camere interne del LiDAR presente nel ramo upstream gz-rendering8, collegato alla PR 1378 [4].

Il caso 2D richiedeva anche di evitare la coordinata verticale non finita generata dalla formula generale con campo verticale nullo. Non viene aggiunto un offset fittizio al robot e non vengono alterate le distanze pubblicate.

```
less sources/gazebo-virgl.patch
```

Puoi leggere la patch prima di proseguire. La correzione si trova nel componente grafico, non nei file di TurtleBot.

Se il controllo SHA256 o l'applicazione della patch falliscono, fermati. Non usare patch --force. Se una preparazione precedente è rimasta incompleta, usa una nuova copia del kit invece di riutilizzare sorgenti di stato incerto.

7. COMPILARE E ASSEMBLARE IL RUNTIME

```
bash prepare.sh build
bash prepare.sh assemble
```

La compilazione usa due processi paralleli per limitare l’impatto sulla VM. Non serve eseguire colcon su tutta la distribuzione ROS e non serve installare il progetto gz-rendering nel sistema.

La fase assemble crea un runtime autosufficiente in build/runtime con:

- componente OGRE 1 corretto e collegamento con il nome atteso da Gazebo;

- Mesa privata, driver VirGL e dipendenze;

- profilo che seleziona queste librerie solo nei processi Gazebo;

- libreria dei default per il server nativo e per la sincronizzazione GUI del LiDAR;

- sorgenti, manifest con hash e licenze dei pacchetti copiati.

```
ls build/runtime
ls build/runtime/engine
ls build/runtime/mesa/dri
```

Atteso: profile.rb, libgazebo_virgl_defaults.so, le directory engine e mesa; nell’engine libgz-rendering-ogre.so; nei driver virtio_gpu_dri.so.

Il comportamento del profilo è esplicito: OGRE 1/OpenGL, Qt xcb, Mesa privata e rimozione dei comuni override di rendering software. Il caricamento del componente usa un percorso assoluto per evitare che venga scelta prima la copia standard del vendor.

Se assemble segnala che build/runtime esiste già, non sovrascrivere un runtime alla cieca. Per ripetere una procedura su una nuova macchina parti da una nuova cartella del kit.

8. CONTROLLARE E INSTALLARE COME DEFAULT

Chiudi Gazebo e i launch ROS prima dell'installazione, salvando l'eventuale lavoro. Dalla cartella del kit esegui prima il controllo senza modifiche:

```
python3 install.py --check
```

Atteso: runtime e ingressi verificati; nessuna modifica eseguita. Lo script controlla i file originali collaudati e rifiuta un profilo o una diversione già presenti. Se la tua VM era già stata corretta, lo STOP è previsto: non reinstallare.

```
sudo python3 install.py --install
```

L'installazione crea il runtime privato sotto /usr/local/lib/gazebo-virgl-defaults/runtime-v2. Conserva gli ingressi originali con dpkg-divert e installa due deleghe Ruby compatibili con i launch ROS:

```
/opt/ros/jazzy/opt/gz_tools_vendor/bin/gz
/opt/ros/jazzy/lib/ros_gz_sim/gzserver
```

Gli originali diventano gz.vendor e gzserver.vendor. Backup e manifest sono salvati sotto /var/backups/gazebo-virgl-defaults e /usr/local/share/gazebo-virgl-defaults.

Gli hash degli ingressi sono fissati alle versioni collaudate. Se differiscono, lo script si ferma prima delle modifiche. Questo limita la procedura alla combinazione verificata: non significa che sia sicuro aggirare il controllo su altre release.

Non vengono modificati Mesa di sistema, /etc/ld.so.preload, .bashrc, SDF/URDF, mondi o file TurtleBot. Il runtime funziona anche per altri modelli che usano gli stessi ingressi Jazzy/Harmonic, fermo restando il collaudo dei sensori e delle feature effettivamente necessari.

9. VERIFICARE IL DEFAULT GRAFICO

Apri un nuovo terminale Ubuntu. Controlla il renderer nel solo ambiente del profilo Gazebo:

```
cd "$HOME/gazebo-parallels/kit"
bash tools/private_glxinfo.sh
```

Atteso: VirGL, accelerazione attiva e Mesa 24.0.5. Poi esegui glxinfo senza il profilo: deve mostrare la versione Mesa di sistema, che non è stata retrocessa.

```
glxinfo -B
```

Avvia un mondo standard con il comando normale:

```
source /opt/ros/jazzy/setup.bash
gz sim -r shapes.sdf
```

Ruota la camera e osserva la scena per almeno un minuto. Controlla colori, geometrie, assenza di lampeggiamenti e risposta ai comandi della camera. I primi modelli Fuel possono richiedere rete e tempo di scaricamento.

Per osservare il contatore GUI, aggiungi il pannello Camera FPS dal menu dei plugin. Il valore riguarda la camera della GUI; non è il frame rate della camera del robot e non è una misura indipendente dei frame presentati dallo schermo.

Chiudi questa prova prima di avviare TurtleBot. Non lasciare due server non isolati sullo stesso mondo e sugli stessi topic: possono confondere le misure di clock, scansioni e immagini.

10. PROVARE TURTLEBOT, CAMERA E SCANSIONI

TurtleBot è una prova aggiuntiva, non una dipendenza necessaria al funzionamento del profilo. Su una VM nuova installa i pacchetti del modello e un visualizzatore immagini:

```
sudo apt install ros-jazzy-turtlebot3-gazebo ros-jazzy-rqt-image-view
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=waffle
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Non aggiungere opzioni di rendering al launch. TURTLEBOT3_MODEL seleziona il modello richiesto dal pacchetto TurtleBot; non è una variabile del workaround.

In un secondo terminale:

```
source /opt/ros/jazzy/setup.bash
ros2 topic list
ros2 run rqt_image_view rqt_image_view
```

Nel visualizzatore seleziona /camera/image_raw. Devi vedere l'ambiente, non soltanto una finestra o un topic esistente. Controlla /scan con:

```
ros2 topic hz /scan
```

Atteso circa 10 Hz. Per ispezionare contenuto, range e immagine in una cartella di risultati:

```
cd "$HOME/gazebo-parallels/kit"
mkdir -p risultati
cd risultati
python3 ../tools/check_sensors.py
```

Dopo circa 23 secondi vengono salvati sensor-validation.json e turtlebot-camera.png. Nel mondo TurtleBot i ritorni devono variare; tutti i raggi a 0,12 m indicano che il difetto non è risolto. Una scansione tutta fuori portata può invece essere corretta in un mondo vuoto: il criterio dipende dalla scena.

11. VERIFICARE LE DISTANZE CONTRO UNA PARETE NOTA

Chiudi TurtleBot. Il kit contiene una scena con parete frontale a 2 metri e un sensore statico. Usa una partizione dedicata per separare il test dagli altri processi Gazebo.

Terminale A, dalla cartella del kit:

```
source /opt/ros/jazzy/setup.bash
export GZ_PARTITION=gazebo_kit_check
gz sim -r -s "$HOME/gazebo-parallels/kit/tests/known-distance.sdf"
```

Terminale B:

```
source /opt/ros/jazzy/setup.bash
export GZ_PARTITION=gazebo_kit_check
cd "$HOME/gazebo-parallels/kit"
python3 tools/validate_known.py
```

Atteso: passed: true, distanza frontale circa 2 metri e massimo errore inferiore a 0,02 m, la tolleranza di accettazione del test. Nel collaudo il massimo sui 131 raggi controllati è stato circa 0,00147 m. Il JSON viene salvato nella cartella corrente.

Ferma il terminale A con Ctrl+C. Per la prova 3D avvia al suo posto:

```
gz sim -r -s \
  "$HOME/gazebo-parallels/kit/tests/known-distance-3d.sdf"
```

Nel terminale B, con la stessa partizione ancora impostata:

```
python3 tools/validate_known.py --3d
```

Atteso: 1083 raggi e passed: true. Dopo il test chiudi il server e i terminali di prova, oppure esegui unset GZ_PARTITION prima di tornare ai comandi quotidiani.

12. CONTROLLARE LA POSIZIONE DEL VISUALE LIDAR

Nel mondo TurtleBot aggiungi il pannello Visualize Lidar, aggiorna l'elenco dei topic e scegli /scan. I raggi devono partire dal sensore sul robot e seguirne il movimento.

È stato osservato un ulteriore problema della GUI: il pannello non trovava l'entità associata al topic e disegnava il ventaglio vicino all'origine del mondo. Questo è distinto dal calcolo delle distanze.

La revisione 3 della libreria del kit richiede automaticamente a GuiRunner uno stato completo della scena quando selezioni il topic. Non cambia le coordinate del robot, i frame ROS o le misure pubblicate.

Collaudo della correzione: modello di prova spostato a (-2, 1, 0,5), visuale LiDAR rilevato esattamente a (-2, 1, 0,5). Un test separato con sensore anche ruotato ha confermato distanze coerenti con la posa, con errore inferiore a 4 mm sui raggi controllati.

Recupero manuale alternativo, se stai usando una sessione GUI aperta prima dell'aggiornamento o devi verificare il problema:

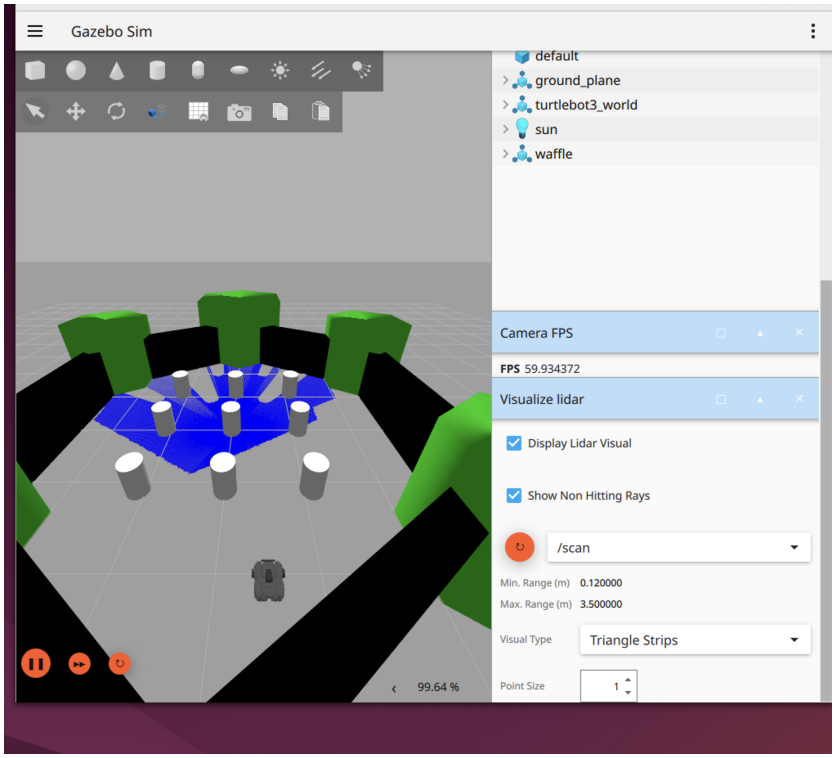
```
source /opt/ros/jazzy/setup.bash
cd "$HOME/gazebo-parallels/kit"
./build/engine/refresh_gui_state default
```

Usa il vero nome del mondo al posto di default se differente. Senza argomento l'utility richiede gli stati dei mondi disponibili. La GUI deve essere già connessa. Questo comando sincronizza lo stato grafico; non sposta il robot nel motore fisico.

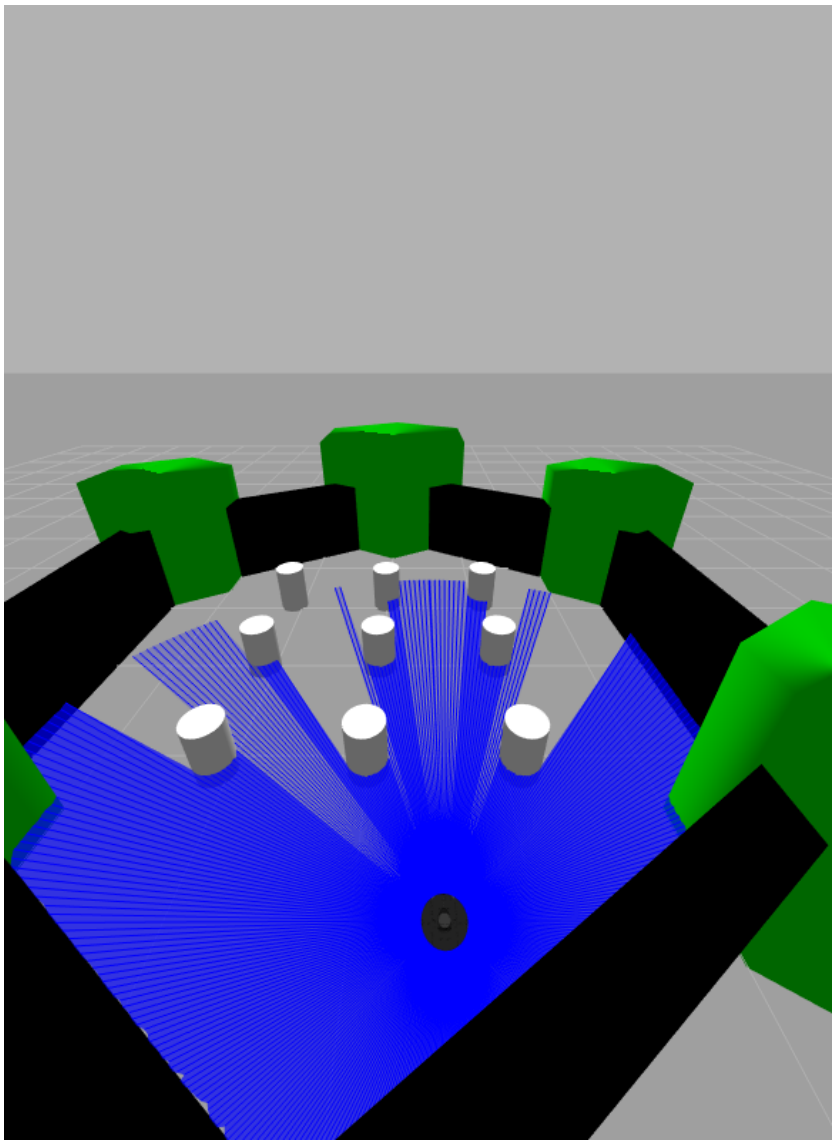
Non compensare il difetto aggiungendo metri alla posa del LiDAR nel file del modello o alterando i frame: introdurresti un errore nei dati che erano già corretti.

IL CASO OSSERVATO: PRIMA E DOPO

Prima / Before



Dopo / After



Prima: il robot è nella parte bassa della scena, mentre il ventaglio blu è vicino al centro del mondo. Il log di Visualize Lidar segnalava che non trovava l'entità del sensore.

Dopo il recupero dello stato completo: il ventaglio blu parte dal robot. Le due immagini hanno inquadrature diverse; il criterio di verifica è l'origine dei raggi sul sensore, non la corrispondenza dei singoli pixel.

Per distinguere visualizzazione e misure, non basta guardare i raggi: servono anche una scena con distanze note e una prova con sensore traslato o ruotato. Queste verifiche sono state eseguite nella diagnosi.

13. PRESTAZIONI REALISTICHE E VALIDITÀ SUGLI ALTRI MODELLI

Controllo sulla VM del caso	Risultato
GUI, 85 finestre di 5 secondi	Mediana 59,95; minimo 55,36 aggiornamenti/s
Camera RGB 1920x1080	Circa 8-11 immagini/s su ROS
LiDAR TurtleBot3	360 raggi, circa 10 Hz
Parete 2D, 131 raggi controllati	Errore massimo circa 1,5 mm
Parete 3D, 183 raggi controllati	Errore massimo circa 0,95 mm
Movimento del robot	Immagini e scansioni cambiano coerentemente

Il rate nominale di 30 Hz della camera Full HD non è stato raggiunto. Non confonderlo con i circa 60 aggiornamenti della GUI. Togliere l'antialiasing non aveva migliorato il caso e non è parte della configurazione adottata.

Per una nuova applicazione valuta separatamente risoluzione, numero di sensori, frequenze, complessità del mondo e richieste dell'algoritmo. Una risoluzione minore può essere una scelta di progetto, ma non viene imposta dal kit ai modelli esistenti.

Le modifiche sono nel runtime e negli ingressi Gazebo, quindi non dipendono da TurtleBot. Sono stati verificati camera RGB, LiDAR orizzontale 2D, un caso LiDAR 3D, avvio ROS e server nativo. Depth/RGB-D, segmentazione, sensori termici, materiali avanzati e altri casi non sono stati collaudati da questa procedura.

Le ombre delle luci restano disabilitate. Se un esperimento richiede proprio quelle caratteristiche o feature specifiche di OGRE 2, questo workaround deve essere valutato rispetto ai requisiti dell'esperimento.

14. USO QUOTIDIANO, AGGIORNAMENTI E RIPRISTINO

Prima di cambiare mondo, ferma il launch precedente con Ctrl+C nel suo terminale e attendi la chiusura dei processi. Nei launch ROS la GUI e il server possono essere separati: chiudere soltanto la finestra non garantisce che il server sia terminato. Una nuova GUI può collegarsi al vecchio server e mostrare ancora TurtleBot, anche se è stato richiesto un altro mondo.

Dopo l'installazione usa gli ingressi normali:

```
source /opt/ros/jazzy/setup.bash
gz sim -r shapes.sdf
```

Oppure il launch ROS del modello scelto, come quello TurtleBot del passo 10. Non serve tenere aperta la cartella del kit o esportare a mano LD_PRELOAD e le variabili Mesa. La shell continua ad avere bisogno del normale ambiente ROS.

Per verificare che il pacchetto TurtleBot non sia stato modificato:

```
dpkg -V ros-jazzy-turtlebot3-gazebo
```

Sulla VM del caso il comando non ha prodotto differenze. Per altri modelli controlla il pacchetto corrispondente o il repository del modello.

Dopo aggiornamenti importanti di ROS, Gazebo, Parallels o Mesa, ripeti le verifiche grafiche e dei sensori. Il runtime privato non viene automaticamente aggiornato da apt. Non applicare questo kit a un'altra versione principale di Gazebo.

Per ripristinare: chiudi prima i launch e Gazebo, poi esegui:

```
sudo /usr/local/sbin/gazebo-virgl-defaults-rollback
```

Lo script rimuove le deleghe gestite e ripristina gli ingressi .vendor tramite dpkg-divert. Non reinstalla Mesa. Il runtime rimane su disco, ma non è più caricato dagli ingressi ripristinati. Apri un nuovo terminale e riavvia Gazebo.

GZ_VIRGL_DEFAULTS=0 consente un opt-out per un singolo processo a scopo diagnostico; non è una soluzione alternativa collaudata. Il rollback completo è stato revisionato, ma non è stato eseguito sulla VM operativa per non annullare la configurazione richiesta.

15. SE UN PASSAGGIO NON FUNZIONA

Sintomo	Controllo e azione
Versione o hash non corrispondente	Fermarsi: il kit non copre quella combinazione. Non rimuovere il controllo.
SDK o header mancanti	Controllare passo 2, source di Jazzy e primo errore CMake. Non copiare librerie da altri OS.
Pacchetto Mesa storico assente	Controllare repository Noble ufficiali e apt update. Non fare downgrade del sistema.
Impossibile aprire il display	Provare nel terminale grafico Ubuntu. Via SSH verificare DISPLAY e autorizzazione della sessione.
Renderer llvmpipe	Controllare VirGL nella VM e differenza fra glxinfo normale e privato.
Camera nera o LiDAR al minimo	Controllare che il processo carichi il runtime privato, non solo l’esistenza dei topic.
Raggi lontani dal robot	Controllare selezione topic e sincronizzazione GUI del passo 12; non spostare il modello.
Si apre sempre il vecchio mondo	Fermare il launch precedente con Ctrl+C e verificare i server rimasti in background. Non basta chiudere la GUI.
Rate o clock strani	Verificare che non ci siano due server sullo stesso mondo/topic. Chiudere le proprie prove.

Per vedere gli avvii correnti senza terminarli:

```
ps -u "$USER" -o pid,args | grep "gz sim"
```

Un processo attivato dal profilo deve mostrare internamente --render-engine seguito dal percorso del componente privato. Il comando digitato dall’utente rimane quello normale.

Per raccogliere un log di una nuova prova:

```
gz sim -v 4 -r shapes.sdf 2>&1 | tee gazebo-prova.log
```

Conserva log, versioni, renderer e risultati dei test. Non servono password SSH o altri segreti per condividere una diagnosi riproducibile.

16. PERCHÉ IL PROFILO FUNZIONA CON I COMANDI NORMALI

La scelta globale passa da due ingressi concreti. Il file gz di gz_tools_vendor resta uno script Ruby: ros_gz_sim può chiamarlo come ruby <percorso-gz> sim ..., quindi una delega shell non rispetterebbe quel contratto. La delega conserva gli argomenti del mondo e avvia l'originale gz.vendor. Solo per il sottocomando sim, esclusi help/versione e opt-out, carica profile.rb e inserisce il percorso assoluto del plugin OGRE e il backend OpenGL. Non contiene un mondo TurtleBot predefinito.

Il profilo seleziona la Mesa privata con LD_LIBRARY_PATH e LIBGL_DRIVERS_PATH per il processo e i discendenti. Imposta Qt xcb e aggiunge libgazebo_virgl_defaults.so a LD_PRELOAD. Non scrive /etc/ld.so.preload: le altre applicazioni del desktop mantengono la Mesa installata. Il nome runtime-v2 è il percorso del layout; il manifest dell'installatore riporta revision 3, che include il recupero del visuale LiDAR.

La delega gzserver copre invece il server nativo di ros_gz_sim, che non necessariamente passa dalla CLI gz. Carica lo stesso ambiente e avvia gzserver.vendor. La libreria intercetta i costruttori C1/C2 di ServerConfig, chiama prima il costruttore vero e imposta i default del motore GUI/server e del backend. C1/C2 sono ingressi distinti dell'ABI C++, non due impostazioni utente. La libreria non renderizza shader e non attiva da sola una GPU virtuale.

Per il percorso CLI il profilo rimuove le opzioni --render-engine e backend elencate in sources/profile.rb e inserisce le proprie: è una selezione imposta del profilo, non solo un suggerimento che lascia prevalere ogni opzione esplicita. Per il server nativo imposta invece i valori iniziali di ServerConfig; un programma che li sovrascrive in seguito richiede una verifica dedicata. Non è possibile promettere la copertura di binari alternativi, container, altre installazioni ROS o applicazioni che caricano direttamente libgz-sim.

Il secondo compito della libreria è separato: dopo VisualizeLidar::OnTopic richiama il gestore reale, trova GuiRunner e accoda RequestState con Qt. Questo recupera lo stato della scena nella GUI. Non corregge una scansione numericamente sbagliata e non cambia TF o la posa del sensore.

Per vedere le diversioni e il manifest, senza modificarli:

```
dpkg-divert --list | grep gazebo
cat /usr/local/share/gazebo-virgl-defaults/manifest.json
ls -l /opt/ros/jazzy/opt/gz_tools_vendor/bin/gz*
ls -l /opt/ros/jazzy/lib/ros_gz_sim/gzserver*
```

La diversione può non contenere la parola gazebo in ogni riga: controlla anche i due percorsi mostrati da ls. La verifica decisiva del runtime è il processo in esecuzione. Dopo aver avviato Gazebo usa ps per identificare il PID del server o della GUI; non usare il PID della sola delega Ruby già terminata.

```
ps -u "$USER" -o pid,ppid,args | grep -E 'gz sim|gzserver|gz-sim'
```

Sostituisci 12345 con il PID reale individuato, poi controlla le mappe:

```
PID=12345
cat "/proc/$PID/maps" | grep -E 'gazebo-virgl|GLX_mesa|virtio_gpu|libgallium'
```

Devono comparire il runtime privato e le librerie grafiche attese. Se non compaiono, controlla quale processo hai scelto e il log: la sola presenza dei file installati non prova che siano caricati. Evita di pubblicare l'intero ambiente del processo, che può contenere dati privati.

17. MOTIVAZIONI AGGIORNATE E LIMITI DEI SINGOLI INTERVENTI

Mesa 24.0.5 privata. È la combinazione che ha dato immagini corrette nei controlli effettuati, non una versione dimostrata ottima tra tutte quelle esistenti. Nell'ablazione, la camera RGB con rumore produceva frame neri con Mesa 25.2.8/VirGL anche spegnendo le ombre. Con Mesa 24/VirGL e llvmpipe 25 le immagini erano valide. Quindi spegnere le ombre da solo non spiega quel difetto della camera. Mesa 24.2 non è stata confrontata; non c'è evidenza per escluderla o preferirla. Le librerie private restano congelate finché non vengono aggiornate deliberatamente.

Ombre disabilitate. La patch del profilo fa ignorare cast_shadows alle luci. È una rinuncia funzionale adottata nel workaround, non una riparazione generale delle ombre. Il risultato combinato è documentato; una registrazione GUI con e senza questa singola modifica, a parità di tutto il resto, resta da fare. I test di pubblicazione delle immagini non misurano il flicker sul display.

LiDAR con tre righe interne. Tre era una scelta pratica: il centro della riga intermedia corrisponde a v=0,5. Non significa che una GPU VirGL non supporti texture a una riga. Nell'ablazione a 53 prove anche due righe e nessun padding funzionano nei casi controllati, dopo le altre correzioni. Il profilo installato conserva il padding storico; non è stato sostituito silenziosamente con la patch della PR.

Correzioni upstream. Il centraggio dei punti sui pixel (#1370) e il ripristino dell'orientamento delle camere (#1378) sono contributi upstream distinti, da riconoscere ai rispettivi autori. La guardia UV evita invece la formula verticale 0/0 quando l'uscita ha una sola riga. I test successivi della PR hanno aggiunto anche la protezione della singola colonna e la scelta delle camere in base alle righe effettive dopo il ricampionamento. Queste aggiunte non sono tutte presenti nel kit globale originale.

Deleghe, preload e dpkg-divert. Servono a rendere automatico il profilo nei comandi ordinari richiesti dall'utente. Non sono necessarie per dimostrare il bug numerico in gz-rendering. Dipendono da percorsi e simboli ABI di Jazzy/Harmonic; aggiornamenti possono romperle. Conservare backup, manifest, controlli hash e procedura di ripristino è parte del metodo descritto, non un dettaglio facoltativo.

Avvio del mondo sbagliato. Il profilo non forza TurtleBot: durante la diagnosi un server di test rimasto attivo faceva collegare la nuova GUI al mondo precedente. Prima di cambiare mondo chiudi il launch e verifica i processi. Termina soltanto PID che hai identificato come appartenenti alla tua prova.

18. ALTERNATIVE E CONTRIBUTI DA TENERE SEPARATI

L'alternativa_overlay_8_2_3/ contiene il colcon overlay della revisione precedente. È stato verificato con cinque prove funzionali brevi sul sistema esistente: LiDAR stretto, 360 gradi, tre righe, scena traslata/ruotata e camere semplice/con rumore. Rispetta le normali ombre SDF e non installa le deleghe globali o il recupero GUI. Il source dell'overlay da solo non rende OGRE 1 il default di tutti i launch. È utile per isolare il rendering e confrontare i driver, ma non sostituisce la configurazione automatica spiegata in questa guida.

Per consultarne i comandi usa il rapporto tecnico 03, capitoli 8-9, assumendo alternativa_overlay_8_2_3/ come radice del kit. Non eseguire l'overlay sopra il vecchio profilo senza isolare l'ambiente: le deleghe globali possono imporre di nuovo motore e Mesa. La migrazione richiede prima il ripristino controllato del profilo, con il consenso del proprietario della VM. Questa revisione documentale non lo ha eseguito.

La PR #1400 è una correzione numerica proposta a gz-rendering, non un installatore del profilo Parallels. Sul commit 9ed780e sono passate le build Jenkins native Linux, macOS e Windows senza fallimenti riportati. Linux ha eseguito i nuovi test; macOS li salta esplicitamente e Windows li disabilita normalmente. Tre workflow GitHub Actions richiedono ancora approvazione dei maintainer. La PR è ora aperta alla review e GitHub ha richiesto il code owner iche033.

La validazione locale della PR comprende 30 processi di test su quattro varianti del codice, due nuove funzioni GTest e 12 configurazioni di scansione. L'eseguibile GPU-ray completo ha dato 11 pass e uno skip previsto per ciascuno dei due backend completi provati. I sorgenti locali usavano rendering8 8.3.0; la PR pubblica riguarda main, con altra ABI. Non copiare le librerie di main nella Jazzy e non assumere che questa PR risolva la camera nera Mesa 25 o tutte le limitazioni OGRE 2.

Per capire e discutere la patch usa 04_SPIEGAZIONE_ORALE_PATCH_IT_EN.txt e il rapporto tecnico bilingue 03. I file della PR e le evidenze sono nello ZIP gazebo_documentation_v3.zip, nella sottocartella documentation_v3/pr/. Non vanno applicati automaticamente sopra sources/gazebo-virgl.patch del kit globale: i due percorsi hanno basi e scopi diversi.

19. COSA REGISTRARE SULLA NUOVA VM

Prima della procedura registra versione di macOS, Parallels e Tools, checksum dell'immagine Ubuntu, vCPU/RAM, risoluzione e scala dello schermo, tipo di sessione, pacchetti ROS/Gazebo e glxinfo. Registra l'installazione nell'ordine dei passi, compresi eventuali STOP e tutti i comandi con sudo. Una seconda VM pulita non è stata ancora completata in questa indagine.

Dopo l'installazione verifica separatamente: renderer e librerie caricate; scena GUI senza flicker osservabile; camera FPS della GUI; immagini vere della camera del robot; /scan e distanze note; posizione del ventaglio sul sensore; un gz sim normale e un ros2 launch normale. Conserva i risultati anche se non raggiungono quelli storici. Esegui infine una sessione prolungata con movimento e carico rappresentativo: i brevi smoke test non certificano stabilità per ore.

Per un confronto onesto riporta risoluzione, mondo, sensori e frequenze richieste insieme alle prestazioni. La misura storica di circa 1,5 mm, il residuo di circa 31 mm della scena stretta dell'ablazione e la tolleranza di 5 mm del nuovo GTest sono tre condizioni diverse. Non usarle come specifica unica di accuratezza.

Questa revisione aggiorna e organizza i documenti. Non ha modificato la VM, reinstallato il profilo, cambiato i modelli o prodotto nuove misure FPS. Gli script runtime del kit sono gli stessi conservati nella guida originale; le nuove evidenze ne chiariscono il contesto.

FONTI, KIT E LIMITI DEL COLLAUDO

[1] Documentazione ufficiale ROS 2 Jazzy: Ubuntu 24.04 e ambiente ROS. Sorgente ufficiale consultata nel repository ros2_documentation:

https://github.com/ros2/ros2_documentation/blob/jazzy/source/Installation/Ubuntu-Install-Debs.rst

[2] Parallels: VirGL/Virtio GPU per VM Linux e verifica nel guest:

https://kb.parallels.com/en/128518

[3] Gazebo: abbinamento Jazzy/Harmonic e installazione ros_gz dal repository ROS:

https://gazebosim.org/docs/harmonic/ros_installation/

[4] Sorgenti gz-rendering fissati dal kit e correzione upstream dell’orientamento del LiDAR:

https://github.com/gazebosim/gz-rendering/tree/gz-rendering8_8.2.3

https://github.com/gazebosim/gz-rendering/pull/1378

Gli altri risultati, comprese le differenze Mesa 24/25, il problema della singola riga, le distanze, i rate e il recupero della GUI, provengono dai test diretti sulla VM del caso. Non sono dichiarazioni generali sulle prestazioni di ogni Mac.

Il kit è stato ricostruito in una cartella temporanea senza usare i vecchi sorgenti della diagnosi; sono stati verificati compilazione, assemblaggio e funzionamento del runtime ricostruito. Non è stata reinstallata da zero una seconda VM Ubuntu: l’installatore generico è stato revisionato e contiene blocchi di versione/hash e gestione delle diversioni.

L’archivio include la copia testuale di questa guida, i sorgenti e gli script. Non contiene i pacchetti Mesa precompilati: prepare.sh li scarica dai repository configurati di Ubuntu. Il controllo SHA256 dei sorgenti Gazebo è fissato nel kit.

Scopo: rendere riproducibile il workaround verificato, con controlli che impediscano di scambiare una finestra fluida o un topic presente per un sensore corretto.